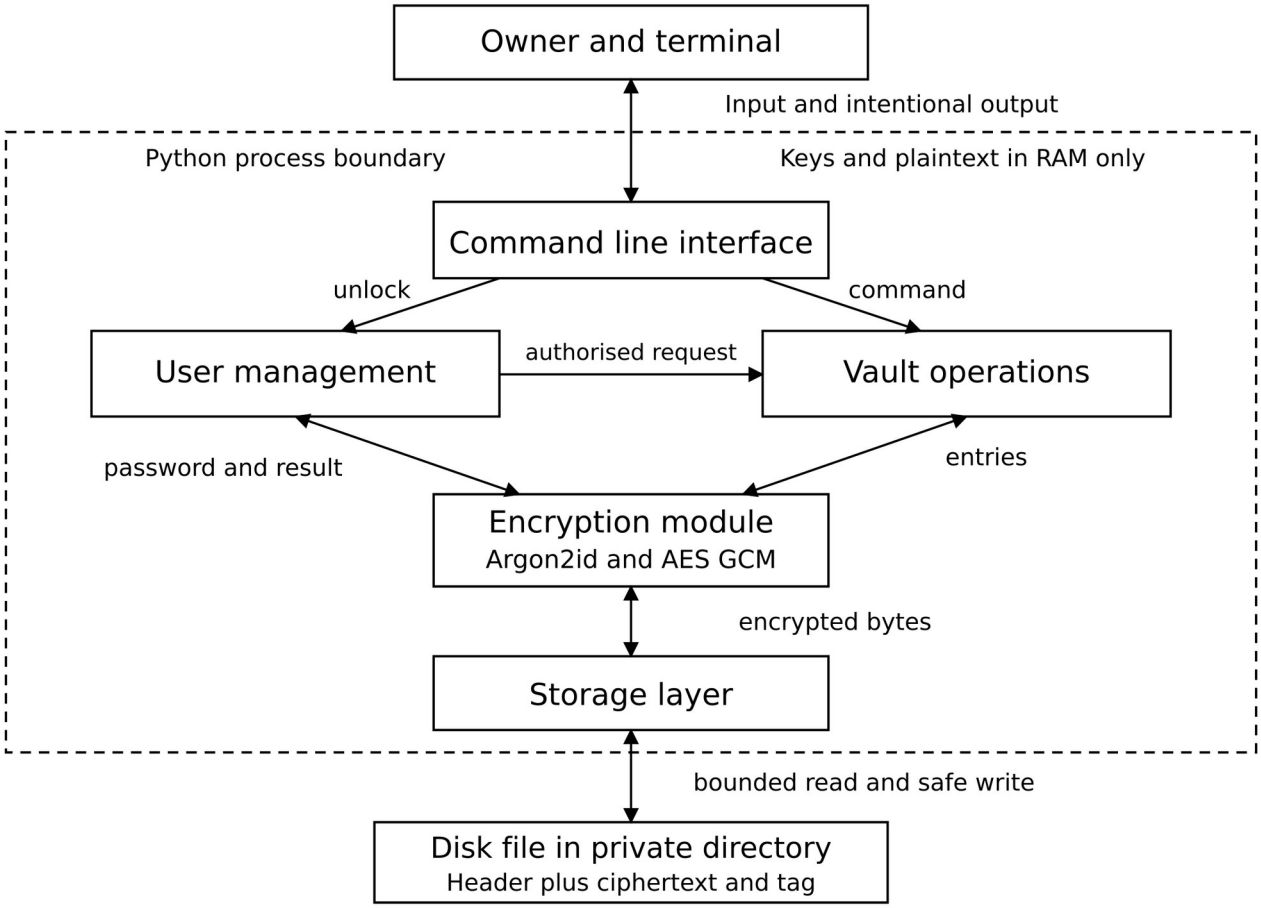
# CLI Password Manager

Secure Programming Checkpoint 1 |Danila Sisov

## 1 Scope and architecture

The project is a local command-line password manager for one user on Linux and macOS. It will store labels, usernames, passwords and notes in one encrypted vault. The owner will create a master password, unlock the vault and manage entries. Each command that accesses an existing vault will authenticate again, then exit. Cloud synchronisation, shared accounts, a web interface and password recovery are outside the initial scope.

I will use Python 3.12 or newer and the cryptography library for Argon2id and AES-GCM. Python keeps the CLI small and readable; the library provides the cryptographic operations. The standard library supplies argument parsing, hidden input, random generation and file handling. Linux and macOS provide the POSIX permissions and file-descriptor APIs used in the design.




User management handles master-password setup, unlock and password changes. It asks the encryption module to derive the key and authenticate the vault. Only a successful unlock permits vault operations. The storage layer handles encrypted bytes and file metadata. The CLI validates commands and presents the selected result.

The terminal and disk are input boundaries: commands and file contents must be checked. Plaintext and keys exist inside the local process only while needed. Modules share one process; their separation is organisational, not memory isolation. The operating system, Python runtime

and installed dependencies are trusted. The program runs as the ordinary owner, without root privileges.

## 2 Vault and cryptographic design

The initial scheme uses Argon2id to derive a 32-byte key from the master password and a random 16-byte salt. Version 1 fixes the parameters at 65536 KiB of memory, 3 passes and 4 lanes. These settings will be measured on the target machine before implementation is finalised. AES-256-GCM encrypts the entire vault payload, using a fresh random 12-byte nonce on every save and the full 16-byte authentication tag.

| Outer field | Stored value |
|---|---|
| version | Integer 1; defines the supported format and AES-256-GCM scheme. |
| kdf | Object with name argon2id, memory_kib 65536, passes 3 and parallelism 4. |
| salt | Base64 encoding of the 16-byte salt; generated at creation or master-password change. |
| nonce | Base64 encoding of a fresh 12-byte random value for this encryption. |
| ciphertext | Base64 encoding of encrypted payload followed by the 16-byte tag. |

The disk file is UTF-8 JSON. Its decrypted payload is an object with an entries array. Each entry has id, label, username, password and notes fields. The id is a UUID in 32-character hex form. All entry fields are encrypted together; only the technical header above remains visible. Base64 is an encoding for binary values, not encryption.

The header consists of every outer field except ciphertext. Its canonical JSON bytes (sorted keys, compact separators and ASCII escaping) are supplied to AES-GCM as associated data. Reject duplicate keys, unexpected fields, invalid types, invalid Base64 and unsupported KDF settings before deriving a key. Verify the tag before parsing or using the decrypted payload.

### Unlock and password changes

A successful tag check proves that the supplied password-derived key opens this vault; no separate password verifier is stored. An incorrect password or damaged protected data produces a generic unlock failure and no write. Never store the master password or derived key. Changing the master password requires the old password, confirmation of the new one, a new salt and nonce, and a complete re-encryption. Forgotten passwords cannot be reset to recover the vault.

### Key lifetime and memory

Keep secrets local to one command and release them in finally blocks; overwrite owned mutable buffers where possible. CPython uses reference counting and cyclic garbage collection, but del and garbage collection do not guarantee secure erasure of immutable strings, bytes or library copies. Disable core dumps for vault operations and minimise copies. Use encrypted swap or disk to limit exposure through OS-managed swap and hibernation. This design does not promise protection from an administrator, a debugger with memory access or malware running as the owner.

## 3 Threat model

Assets are the master password, encryption key, saved credentials and vault integrity and availability. Relevant actors include a thief with a copied vault, another local user and someone supplying malicious input. Accidental interruption is also considered. Priorities are initial design judgements: High means direct secret exposure, authentication failure or corruption; Medium means a more limited or contextual risk.

| Priority | Area and threat | Intended mitigation |
|---|---|---|
| T1<br>High | Master password<br>Offline guessing of a stolen vault. | Require at least 15 characters at creation, encourage a long unique passphrase, and use salted Argon2id. Local retry limits cannot stop offline guessing. |
| T2<br>High | Master password<br>Exposure through arguments, input echo or logs. | Read secrets through getpass in a terminal; abort if input may echo. Never accept passwords as arguments or log their values. |
| T3<br>High | Vault at rest<br>Credentials exposed by copied files or temporary files. | Encrypt all entry fields; use a private directory and owner-only files. Temporary files and backup copies must contain only encrypted data. |
| T4<br>High | Vault at rest<br>Modified header or ciphertext bypasses authentication. | Authenticate the header and payload with AES-GCM. Fail closed on a bad tag or unsupported format; reveal nothing and leave the file unchanged. |
| T5<br>High | File interface<br>Traversal, links or a race redirect file access. | Use fixed filenames in a verified private directory. Open without following symlinks and check the opened descriptor, including owner, type, mode and hard-link count. |
| T6<br>High | Vault at rest<br>An interrupted write or two commands corrupt the vault. | Lock a separate persistent lock file. For updates, fully write and sync encrypted temporary data before atomic replacement; never truncate the working vault first. |
| T7<br>High | Vault in memory<br>A dump or leftover object exposes a key or plaintext. | Use short command lifetimes, limited copies, mutable-buffer cleanup and disabled core dumps. Python cannot guarantee wiping every copy; privileged memory access remains a limitation. |
| T8<br>High | Interface<br>Malformed or huge input exhausts resources or becomes code. | Bound file and field sizes, require the expected JSON schema and fixed KDF settings, reject control characters, and never use shell execution, eval or pickle. |
| T9<br>Medium | Interface<br>Unintended password display or entry deletion. | List IDs and labels only. Reveal a password only through an explicit show in an interactive terminal. Require confirmation before deletion; suppress secret-bearing error details. |
| T10<br>Medium | Vault at rest<br>Deletion or replacement with an older valid copy. | Restrict write permissions and keep user-managed encrypted backups. A self-contained vault cannot reliably detect rollback; losing every copy means losing the data. |

Misuse examples: an attacker copies vault.json and tries guesses offline; the intended defence is the master passphrase and Argon2id, not the CLI retry policy (T1). An attacker replaces vault.json with a symlink before it is opened; the intended result is refusal without touching the target (T5). Both scenarios become tests in the implementation stage.

## 4 File handling and repository plan

Use ~/.securepm with mode 0700 and vault.json, temporary files, logs and a persistent lock file with mode 0600. Verify the directory and its parents, anchor operations to its open descriptor, and use O_NOFOLLOW and fstat. Accept only owner-owned regular files with one hard link and no group or other access. Entry labels never become filenames. New-vault creation uses O_CREAT with O_EXCL, so it cannot overwrite an existing vault.

Hold a separate advisory lock from read through completion. For an existing vault, create a unique temporary file in the same directory, write only encrypted data, flush and fsync it, replace the vault atomically, then sync the directory. Keep the lock file in place across replacements. On an error before replacement, preserve the old vault and remove the temporary file when possible. Handle Ctrl+C and EOF without exposing secrets; never report success before the write is complete.

Initial limits are an 8 MiB outer file, 500 entries, 128 UTF-8 bytes for labels, 256 for usernames, 1024 for master and stored passwords and 4096 for notes. Check limits before use and reject terminal control characters. Do not trim or silently truncate passwords. Logs contain only time, operation and outcome, with no entry contents, password values or keys.

### Planned commands

| Command | Purpose |
|---|---|
| init | Create the vault and confirm a master password. |
| add<br>list | Add an entry through prompts; list IDs and labels after unlock. |
| show ENTRY_ID | Unlock and intentionally reveal the selected entry in a terminal. |
| update ENTRY_ID<br>delete ENTRY_ID | Unlock; edit an entry or confirm its deletion. |
| change-master | Unlock with the old password and re-encrypt with the new one. |
| generate --length 20 | Generate with secrets; default 20, allowed 12 to 128 characters. |